ShakeBox

GS805 COFFEE MACHINE

Interface Guide

Flutter SDK (Dart)

Version 0.1.3

Date 2026-04-08

Author Anyeats World Co., Ltd.

Contact rnd@anyeats.co.kr

Classification Confidential

문서 이력

버전	날짜	내용
0.1.4	2026-04-10	주문-제작-완성 흐름 추가
0.1.3	2026-04-08	0x15 레시피 시간 설정 추가, 미지원 명령 제거 (0x1D, 0x25, 0x1B, 0x24)
0.1.2	2026-04-06	상태 코드 레퍼런스 추가
0.1.1	2026-04-02	Hardware UART (UartSerialConnection) 연결 방식 추가
0.1	2026-04-01	초판 작성 — 전체 API (기본 14개 + 능동보고 4개 + R시리즈 2개 + Series 3/R 확장 11개)

목차

1. 개요

1.1 통신 사양

1.2 지원 모델

1.3 지원 플랫폼

1.4 아키텍처

2. 설치 및 설정

2.1 패키지 추가

2.2 Android 설정

3. 빠른 시작

4. API 레퍼런스

4.1 연결 관리

4.2 음료 제조 및 제어

4.3 정보 조회

4.4 유지보수

4.5 Series 3/R 확장 명령

4.6 레시피 시간 설정 (0x15)

4.7 Event Streams

4.8 Properties

4.9 주요 클래스

5. 사용 예제

5.1 연결 관리

5.2 음료 제조

5.3 온도 설정

5.4 상태 및 에러 조회

5.5 레시피 시간 설정 + 음료 제조

5.6 이벤트 스트리밍

5.7 자동 재연결

6. 기술 사양

7. 주문-제작-완성 흐름

7.1 상태 확인 명령

7.2 결제 포함 전체 흐름

7.3 실패 시 흐름

8. 상태 코드 레퍼런스

8.1 STA Code (응답 상태 코드)

8.2 Controller Status (0x1E)

8.3 Drink Status (0x1F)

8.4 능동 보고 이벤트

1. 개요

본 문서는 ShakeBox(GS805) 커피머신과 Flutter 앱 간 시리얼 통신을 위한 **GS805Serial Flutter SDK**의 인터페이스 가이드입니다. SDK는 Dart Stream/Future 기반으로 설계되어 비동기 통신과 실시간 이벤트 스트리밍을 지원합니다.

1.1 통신 사양

항목	사양
통신 방식	UART RS232 (USB Serial / Hardware UART)
전송 속도	9600 bps
데이터 형식	8N1 (8 data bits, No parity, 1 stop bit)
바이트 순서	Big Endian
응답 타임아웃	100ms (재시도 메커니즘 내장)

1.2 지원 모델

분류	시리즈	제어 시스템	해당 모델
Series 2	2nd	현마 (단일)	GS305, GS505, JK81, JK90, JK91
Series 3	3rd	**단일 스레드**	**GS505, GS805**, JK81, JK82, JK86
R Series	3rd	멀티 스레드	GS801, RD-JK86, JK88

> **참고:** ShakeBox(GS805)는 Series 3 (단일 스레드)입니다. 본 문서는 GS805에서 동작 확인된 명령만 기술합니다.

1.3 지원 플랫폼

플랫폼	지원 상태
Android	지원 (API 19+, Android 4.4+)
iOS	제한적 (USB Serial 제약)

1.4 아키텍처

```
┌──────────────────────────────────┐
│      API Layer (GS805Serial)     │  ← 고수준 사용자 API
```

```
├──────────────────────────────────┤
│   Serial Layer (SerialManager)   │  ← 연결 및 메시지 파싱
├──────────────────────────────────┤
│  Protocol Layer (GS805Protocol)  │  ← 메시지 인코딩/디코딩
└──────────────────────────────────┘
```

2. 설치 및 설정

2.1 패키지 추가

Git 의존성 (권장)

```yaml
# pubspec.yaml
dependencies:
  gs805serial:
    git:
      url: https://github.com/anyeats/anyeats-mip-sdk-dart.git
      ref: 1.0.0
```

```
# 패키지 설치
$ flutter pub get
```

2.2 Android 설정

`AndroidManifest.xml` 에 USB 권한을 추가합니다.

```xml
<!-- AndroidManifest.xml -->
<manifest>
  <uses-feature android:name="android.hardware.usb.host" />

  <application>
    <intent-filter>
      <action android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED" />
    </intent-filter>

    <meta-data
      android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
      android:resource="@xml/device_filter" />
  </application>
</manifest>
```

`android/app/src/main/res/xml/device_filter.xml` 생성:

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <usb-device vendor-id="1234" product-id="5678" />
</resources>
```

기술 스택

항목	사양
언어	Dart
비동기	Stream / Future
시리얼 통신	USB: usb_serial 패키지 / UART: JNI (termios) + Platform Channel
Flutter SDK	>= 3.0.0
Dart SDK	>= 3.0.0
최소 Android	API 19 (Android 4.4)

3. 빠른 시작

```dart
import 'package:gs805serial/gs805serial.dart';

void main() async {
  // 1. 인스턴스 생성 (Hardware UART, 자동 재연결 포함)
  final gs805 = GS805Serial(
    connection: UartSerialConnection(),
    reconnectConfig: ReconnectConfig.exponentialBackoff,
  );

  // 2. 디바이스 검색 및 연결
  final devices = await gs805.listDevices();
  await gs805.connect(devices.first);

  // 3. 상태 조회
  final status = await gs805.getMachineStatus();
  print('Status: ${status.description}');

  // 4. 음료 제조
  await gs805.makeDrink(DrinkNumber.hotDrink1);

  // 5. 이벤트 관찰 (Stream)
  gs805.eventStream.listen((event) {
    print('Machine event: ${event.type.description}');
  });

  // 6. 연결 해제
  await gs805.disconnect();
  await gs805.dispose();
}
```

4. API 레퍼런스

4.1 연결 관리

메서드	설명
`listDevices()` → `Future<List<SerialDevice>>`	시리얼 디바이스 목록 반환 (USB 또는 UART)
`connect(SerialDevice device)` → `Future<void>`	지정 디바이스에 연결
`connectToFirstDevice()` → `Future<void>`	첫 번째 디바이스에 자동 연결
`connectByVidPid(int vid, int pid)` → `Future<void>`	VID/PID를 지정하여 연결
`disconnect()` → `Future<void>`	현재 연결 해제
`reconnect()` → `Future<void>`	수동 재연결 트리거

4.2 음료 제조 및 제어

메서드	설명
`makeDrink(DrinkNumber, {bool useLocalBalance})` → `Future<void>`	음료 제조 명령 전송
`setHotTemperature(int upper, int lower)` → `Future<void>`	온수 온도 설정 (60~99 °C)
`setColdTemperature(int upper, int lower)` → `Future<void>`	냉수 온도 설정 (2~40 °C)
`setCupDropMode(CupDropModeEnum mode)` → `Future<void>`	컵 낙하 모드 설정
`setDrinkPrice(DrinkNumber, int price)` → `Future<void>`	음료 가격 설정 (0~99)

4.3 정보 조회

메서드	설명
`getMachineStatus()` → `Future<MachineStatus>`	머신 상태 조회 (ready, busy, soldOut 등)
`getErrorCode()` → `Future<MachineError>`	에러 코드 비트 플래그 조회
`getErrorInfo()` → `Future<ErrorInfo>`	에러 상세 정보 + 복구 가이드

메서드	설명
`getBalance()` → `Future<MachineBalance>`	투입 잔액 조회
`getSalesCount(DrinkNumber)` → `Future<DrinkSalesCount>`	음료별 판매 수량 조회

4.4 유지보수

메서드	설명
`testCupDrop()` → `Future<void>`	컵 낙하 테스트
`autoInspection()` → `Future<void>`	자동 점검 실행
`cleanAllPipes()` → `Future<void>`	전체 파이프 세척
`cleanSpecificPipe(int number)` → `Future<void>`	특정 파이프 세척 (번호 지정)
`returnChange()` → `Future<ChangerStatus>`	거스름돈 반환

4.5 Series 3/R 확장 명령

> 아래 명령은 Series 3 및 R시리즈 모델에서 사용 가능합니다. Series 2 모델에서는 지원되지 않습니다.

CMD	메서드	설명
단위 기능 테스트		
0x1A	`unitFunctionTest(int testCmd, int data1, int data2, int data3)` → `Future<void>`	분말/온수/좌표/컵/뚜껑/문/얼음 등 개별 기능 테스트
	문 제어: 열기 `unitFunctionTest(3, 4, 0, 0)` / 닫기 `unitFunctionTest(3, 4, 1, 0)`	
물 보충		
0x1C	`waterRefill()` → `Future<void>`	물 보충 작업 실행
상태 조회 (상세)		
0x1E	`getControllerStatus()` → `Future<ControllerStatus>`	메인 컨트롤러 32비트 상세 상태 조회
0x1F	`getDrinkStatus()` → `Future<DrinkPreparationStatus>`	음료 제작 진행률 및 실패 코드 (EM01~EM26)
에러 조회 (객체별)		
0x22	`getObjectException(ObjectType)` → `Future<ObjectExceptionInfo>`	12가지 객체별 상세 에러 정보 조회
강제 종료		
0x23	`forceStopDrinkProcess()` → `Future<void>`	음료 제조 프로세스 강제 종료
	`forceStopCupDelivery()` → `Future<void>`	컵 전달 프로세스 강제 종료

4.6 레시피 시간 설정 (0x15)

채널별 분말 시간과 물 시간을 설정하여 음료 레시피를 구성합니다.

CMD	메서드	설명
0x15	`setDrinkRecipeTime(DrinkNumber, List<ChannelTime>)` → `Future<void>`	음료별 채널 시간 설정 (8채널, 0.1초 단위)

파라미터

파라미터	값	설명
`drink`	0x01~0x07 (온음료), 0x11~0x17 (냉음료)	음료 번호
`channelTimes`	8채널 × (materialTime, waterTime)	각 채널의 분말/물 시간 (0.1초 단위)

채널 매핑

채널	설명	비고
ch1	물 + 1번통	동시 불가 (물과 분말 순차 출력)
ch2	2번통	
ch3	3번통	
ch4	4번통	
ch5~ch8	확장 채널	

> **참고:** 채널은 순차적으로 실행됩니다 (재정렬 없음). 값이 (0, 0)인 채널은 비활성화됩니다.

4.7 Event Streams

SDK는 Dart Stream 기반의 실시간 이벤트 스트리밍을 제공합니다.

Stream	타입	설명
`messageStream`	`Stream<ResponseMessage>`	모든 응답 메시지 (raw)
`eventStream`	`Stream<MachineEvent>`	머신 이벤트 (능동 보고)
`connectionStateStream`	`Stream<bool>`	연결 상태 변화
`reconnectEventStream`	`Stream<ReconnectEvent>`	자동 재연결 이벤트

능동 보고 이벤트 (MachineEventType)

CMD	이벤트	설명
0x05	`cupDropSuccess`	컵 낙하 성공
0x10	`drinkComplete`	음료 제조 완료
0x06	`iceDropComplete`	얼음 투하 완료
0x20	`trackObstacle`	트랙 장애물 감지

4.8 Properties

속성	타입	설명
`isConnected`	`bool`	현재 연결 여부
`connectedDevice`	`SerialDevice?`	연결된 디바이스 정보
`isReconnecting`	`bool`	재연결 진행 중 여부
`bufferSize`	`int`	수신 버퍼 크기

4.9 주요 클래스

클래스	설명
`GS805Serial`	메인 API 클래스
`SerialDevice`	시리얼 디바이스 정보 (USB / UART)
`DrinkNumber`	음료 선택 enum (14종)
`MachineStatus`	머신 상태 enum (9종)
`MachineError`	에러 비트 플래그 정보

클래스	설명
`MachineEvent`	능동 보고 이벤트
`ControllerStatus`	32비트 컨트롤러 상태 (Series 3/R)
`DrinkPreparationStatus`	음료 제작 진행률/실패 (Series 3/R)
`ObjectExceptionInfo`	객체별 에러 상세 (Series 3/R)
`ObjectType`	에러 조회 대상 객체 enum (Series 3/R)
`ChannelTime`	채널별 분말/물 시간 (0x15)
`ReconnectConfig`	자동 재연결 설정

5. 사용 예제

5.1 연결 관리

```
final gs805 = GS805Serial();

// 디바이스 목록 조회
final devices = await gs805.listDevices();

// 특정 디바이스에 연결
await gs805.connect(devices.first);

// VID/PID로 연결
await gs805.connectByVidPid(0x1234, 0x5678);

// 연결 상태 확인
if (gs805.isConnected) {
  print('Connected to: ${gs805.connectedDevice?.name}');
}

// 연결 해제
await gs805.disconnect();
```

5.2 음료 제조

```
// 온음료 1번 제조
await gs805.makeDrink(DrinkNumber.hotDrink1);

// 냉음료 3번 제조
await gs805.makeDrink(DrinkNumber.coldDrink3);

// 로컬 잔액 사용
await gs805.makeDrink(
  DrinkNumber.hotDrink2,
  useLocalBalance: true,
);

// 사용 가능한 음료 목록
DrinkNumber.hotDrinks;   // [hotDrink1 ~ hotDrink7]
DrinkNumber.coldDrinks;  // [coldDrink1 ~ coldDrink7]
```

5.3 온도 설정

```
// 온수 온도 설정 (60~99°C)
await gs805.setHotTemperature(
  upperLimit: 85,
  lowerLimit: 75,
);

// 냉수 온도 설정 (2~40°C)
await gs805.setColdTemperature(
  upperLimit: 10,
  lowerLimit: 5,
);
```

5.4 상태 및 에러 조회

```
// 머신 상태 조회
final status = await gs805.getMachineStatus();
print('Status: ${status.description}');

// 잔액 조회
final balance = await gs805.getBalance();
print('Balance: ${balance.balance} tokens');

// 판매 수량 조회
final sales = await gs805.getSalesCount(DrinkNumber.hotDrink1);
print('Local: ${sales.localSalesCount}');
print('Command: ${sales.commandSalesCount}');

// 에러 상세 정보 + 복구 가이드
final errorInfo = await gs805.getErrorInfo();
if (errorInfo.error.hasError) {
  print('Severity: ${errorInfo.severity}');
  for (var action in errorInfo.recoveryActions) {
    print('  - $action');
  }
}
```

5.5 레시피 시간 설정 + 음료 제조

```
// 1. 인스턴스 생성 (Hardware UART)
final gs805 = GS805Serial(
  connection: UartSerialConnection(),
);
final devices = await gs805.listDevices();
await gs805.connect(devices.first);

// 2. 레시피 시간 설정 (0x15) — 온음료 1번
// ch1: 분말 3.0초 + 물 5.0초, ch2: 분말 2.0초 + 물 3.0초, 나머지 비활성
await gs805.setDrinkRecipeTime(
  DrinkNumber.hotDrink1,
  [
    ChannelTime(materialTime: 30, waterTime: 50),  // ch1: 3.0s, 5.0s
    ChannelTime(materialTime: 20, waterTime: 30),  // ch2: 2.0s, 3.0s
    ChannelTime(materialTime: 0, waterTime: 0),    // ch3: 비활성
    ChannelTime(materialTime: 0, waterTime: 0),    // ch4: 비활성
    ChannelTime(materialTime: 0, waterTime: 0),    // ch5: 비활성
    ChannelTime(materialTime: 0, waterTime: 0),    // ch6: 비활성
    ChannelTime(materialTime: 0, waterTime: 0),    // ch7: 비활성
    ChannelTime(materialTime: 0, waterTime: 0),    // ch8: 비활성
  ],
);

// 3. 음료 제조
await gs805.makeDrink(DrinkNumber.hotDrink1);

// 4. 문 제어 (unitFunctionTest 사용)
await gs805.unitFunctionTest(3, 4, 0, 0);  // 문 열기
await gs805.unitFunctionTest(3, 4, 1, 0);  // 문 닫기
```

5.6 이벤트 스트리밍

```
// 머신 이벤트 수신
gs805.eventStream.listen((event) {
  switch (event.type) {
    case MachineEventType.cupDropSuccess:
      print('컵 낙하 성공');
      break;
    case MachineEventType.drinkComplete:
      print('음료 완성!');
      break;
    case MachineEventType.iceDropComplete:
      print('얼음 투하 완료');
      break;
    case MachineEventType.trackObstacle:
      print('트랙 장애물 감지');
      break;
  }
});

// 연결 상태 모니터링
gs805.connectionStateStream.listen((isConnected) {
  print(isConnected ? 'Connected' : 'Disconnected');
});

// 재연결 이벤트 모니터링
gs805.reconnectEventStream.listen((event) {
  print('Reconnect: ${event.state}');
  print('Attempt: ${event.attempt}/${event.maxAttempts}');
});
```

5.7 자동 재연결

```
// Exponential Backoff 프리셋
final gs805 = GS805Serial(
  reconnectConfig: ReconnectConfig.exponentialBackoff,
);

// 커스텀 재연결 설정
final customConfig = ReconnectConfig(
  strategy: ReconnectStrategy.exponentialBackoff,
  maxAttempts: 10,
  initialDelay: Duration(milliseconds: 500),
  maxDelay: Duration(seconds: 30),
  backoffMultiplier: 2.0,
);

final gs805 = GS805Serial(reconnectConfig: customConfig);

// 재연결 상태 확인
if (gs805.isReconnecting) {
  print('Reconnecting...');
}

// 수동 재연결
await gs805.reconnect();
```

6. 기술 사양

항목	사양
언어	Dart 3.0+
프레임워크	Flutter 3.0+
비동기 처리	Dart Stream / Future
시리얼 라이브러리	USB: usb_serial 패키지 / UART: JNI (termios) + Platform Channel
최소 Android SDK	API 19 (Android 4.4 KitKat)
통신 프로토콜	UART RS232, 9600 bps, 8N1, Big Endian
자동 재연결	Exponential Backoff / Fixed Interval
테스트 커버리지	97+ unit tests
패키지	`gs805serial`

7. 주문-제작-완성 흐름

주문 접수부터 음료 완성까지의 전체 상태 추적 흐름입니다.

7.1 상태 확인 명령

단계	명령	확인값
주문 접수	makeDrink() 응답	STA=0x00 (성공)
제작 시작	getMachineStatus() (0x0B)	Busy (0x01)
제작 진행	getDrinkStatus() (0x1F)	result=Executing(1), step X/Y
컵 배치	getDrinkStatus() Bit6	0=미배치, 1=배치됨
제작 완료	getDrinkStatus() Bit30-31	Success(2)
완성 이벤트	eventStream 능동 보고	DRINK_COMPLETE (0x10)
컵 회수 대기	getDrinkStatus() Bit7	1=회수 대기 중
제작 실패	getDrinkStatus() Bit30-31	Failure(3) + EM코드
대기 복귀	getMachineStatus()	Ready (0x00)

7.2 결제 포함 전체 흐름

```
1. MDB: setup() → enable() (최초 1회)
2. MDB: requestVend(price: 100) → 카드리더 결제 대기 화면
3. 사용자: 카드 태그
4. MDB: vendApproved 이벤트 수신
5. GS805: makeDrink(DrinkNumber.hotDrink1) → STA=0x00
6. GS805: getMachineStatus() → Busy
7. GS805: getDrinkStatus() 폴링 → Executing (step 진행)
8. GS805: eventStream → DRINK_COMPLETE 수신
9. MDB: vendSuccess(itemNumber: 1) → sessionComplete()
10. GS805: getMachineStatus() → Ready
```

7.3 실패 시 흐름

```
1. GS805: getDrinkStatus() → Failure(3) + EM코드
2. MDB: vendCancel() → sessionComplete()
3. 사용자에게 실패 사유 표시 (EM01~EM26)
```

> **TIP:** getDrinkStatus()를 주기적으로 폴링하면 실시간 진행률(step/totalSteps)을 확인할 수 있습니다.

8. 상태 코드 레퍼런스

GS805 프로토콜에서 사용되는 주요 상태 코드 및 플래그 정의입니다.

8.1 STA Code (응답 상태 코드)

모든 응답 메시지에 포함되는 STA 필드로, 명령 실행 결과를 나타냅니다.

코드	이름	설명
`0x00`	SUCCESS	정상/성공, 명령 수락 및 실행
`0x01`	BUSY	장치 사용 중, 명령 실행 불가
`0x02`	ERROR	장치 오류/실행 실패
`0x03`	PARAMETER_ERROR	매개변수 오류
`0x04`	INSUFFICIENT_BALANCE	잔액 부족
`0x05`	CONFIG_RESTRICTED	매개변수 명령 기능 제한
`0x06`	DATA_NOT_FOUND	데이터/레시피 미존재 또는 유효하지 않음
`0x07`	CONDITIONS_NOT_MET	실행 조건 미충족 (예: 제빙기 오프라인, 아이스 음료 제작 요청)
`0x08`	DEVICE_NOT_FOUND	지정 장치/부품/기능 미존재
`0x7F`	ACTIVE_REPORT	제어판이 능동적으로 보고하는 데이터 내용

8.2 Controller Status (0x1E) - ST_INFO 32-bit Flags

`getControllerStatus()` 응답의 32비트 상태 플래그입니다.

Overall Status (Bit 0-1)

값	상태	설명
0	IDLE	정상 (유휴)
1	BUSY	정상 (바쁨)
2	EXCEPTION	예외
3	FAILURE	고장

Module Online/Offline Status

Bit	모듈	0	1	에러 코드
2	전면 도어 모듈	온라인	오프라인	E14
3	제빙 모듈	온라인	오프라인	E15
4	분쇄(Grinding) 모듈	온라인	오프라인	E10

Module Fault Status

Bit	모듈	0	1
5	전면 도어 상태	정상	고장
6	제빙 상태	정상	고장
7	분쇄 상태	정상	고장

Cup / Lid / Material Status

Bit	항목	0	1	에러 코드
8	컵 없음 플래그	컵 있음	컵 없음	E02
9	뚜껑 없음 플래그	뚜껑 있음	뚜껑 없음	A01
10	물탱크 부족	물 있음	물 없음	E01
11	폐수탱크 경고	정상	경고	
12	시동 가열 플래그	시동 시 가열 안 함	첫 가열 중	
13	컵 홀더에 컵	컵 없음	컵 있음	A09
14	콩 호퍼	콩 있음	비어있음	A165
15	주 제어 센서	정상	비정상	E04

Water Tank Status (Bit 16-19)

Bit	탱크	0	1	에러 코드
16	1번 물탱크	물 있음	물 없음	A10
17	2번 물탱크	물 있음	물 없음	A11
18	3번 물탱크	물 있음	물 없음	A12
19	4번 물탱크	물 있음	물 없음	A13

Pump Status (Bit 20-24)

Bit	펌프	0	1	에러 코드
20	1번 펌프	정상	고장	A14
21	2번 펌프	정상	고장	A15
22	3번 펌프	정상	고장	A16
23	4번 펌프	정상	고장	A17
24	부스터 펌프	정상	고장	E19

Tank / Valve / Hardware Status (Bit 25-31)

Bit	항목	0	1	에러 코드
25	온수탱크	정상	가열 시간 초과	A05
26	냉수탱크	정상	냉각 시간 초과	A06
27	배수 솔레노이드 밸브	정상	고장	E20
28	컵 드로퍼 1	정상	고장	E22
29	컵 드로퍼 2	정상	고장	E23
30	트랙 오류	없음	오류 있음	E12
31	전자 잠금 장치	정상	결함	A116

8.3 Drink Status (0x1F) - DK_INFO

`getDrinkStatus()` 응답의 음료 제조 상태 정보입니다.

Execution Result (Bit 30-31)

값	결과	설명
0	NOT_EXECUTED	미실행 (음료 제조 미진행)
1	EXECUTING	실행 중 (음료 제조 실행 중)
2	SUCCESS	성공적으로 완료됨
3	FAILURE	실패로 종료됨

Failure Cause (Bit 24-29) - EM Codes

코드	이름	설명
0	NONE	실패 없음

코드	이름	설명
1	EM01	레시피 정보 이상
2	EM02	컵 시작 위치 전달 실패
3	EM03	컵 작업 위치 전달 실패
4	EM04	컵 종점 위치 전달 실패
5	EM05	문 열기 작동 실패
6	EM06	문 닫힘 작동 실패
7	EM07	자동 컵 할당 실패
8	EM08	수동 컵 배치 시간 초과
9	EM09	저수위 (유량계)
10	EM10	병렬 분쇄 작업 실패 (제작 시)
11	EM11	분쇄 작업 실패 (컵 전달 전)
12	EM12	컵 수령 대기 시간 초과
13	EM13	분쇄 오프라인 또는 결함
14	EM14	제조 과정 강제 종료됨
15	EM15	제빙 실패
16	EM16	뚜껑 배치 실패
17	EM17	뚜껑 압착 실패
18	EM18	전면 도어 오프라인 (제작 전)
19	EM19	전면 도어 장기 바쁨 (제작 전)
20	EM20	분쇄 완료 알림 시간 초과
21	EM21	제조 과정 중 컵 분실
22	EM22	컵 전달 실행 (제작 전 컵 받침에 컵 있음)
23	EM23	분쇄 사전 작동 실패 (컵 할당 후)
24	EM24	인스턴트 단계 실행 실패 (비정상 매개변수 또는 솔레노이드 밸브 오류)
25	EM25	폐수 트레이 수축 실패 (JK86-B)
26	EM26	폐수 트레이 확장 실패 (JK86-B)

8.4 능동 보고 이벤트

머신이 자동으로 보고하는 이벤트입니다 (STA = `0x7F`).

CMD	이벤트	설명
`0x05`	CUP_DROP_SUCCESS	컵 낙하 성공
`0x10`	DRINK_COMPLETE	음료 제조 완료
`0x06`	ICE_DROP_COMPLETE	얼음 투하 완료
`0x20`	TRACK_OBSTACLE	트랙 장애물 감지 / 캡핑 실패 / 전면 도어 오프라인

> **참고:** 상태 코드의 전체 상세 내용은 GS805 프로토콜 문서를 참조하십시오. SDK의 응답 객체는 이 코드들을 파싱하여 타입 안전한 enum으로 제공합니다.

© 2026 Anyeats World Co., Ltd. All rights reserved. — Confidential